Lab S05: Mexi Banco con Docker Compose

Lo de la clase, en tu máquina

Sistemas Distribuidos (Semestre 2027-1, FI-UNAM)

Victor Emiliano Calderón Gutiérrez

27 de septiembre de 2026

Corrección con capturas: 17 de octubre de 2026

Resumen

Este reporte documenta la ejecución sistemática de los seis pasos que conforman la práctica oficial del Laboratorio S05 de Sistemas Distribuidos (*Mexi Banco v05.1*). Se evalúa la orquestación local con Docker Compose, el diagnóstico de estado operativo mediante sondas de salud (*healthchecks*), la ejecución completa de la colección oficial de Postman (apertura de cuentas, transferencias atómicas, idempotencia en SPEI, consultas de auditoría y manejo de errores controlados), el ciclo de vida del estado en volúmenes frente a reinicios y paros destructivos, el estudio arquitectónico de las cuatro capas del monolito, y el análisis técnico línea por línea de los archivos `docker-compose.yml` y `Dockerfile`.

1. Paso 1: Clonación del Repositorio y Preparación del Entorno

Conforme a la especificación oficial del laboratorio, se clonó la etiqueta de referencia `v05.1` del proyecto *Mexi Banco*:

```
git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
cd mexi-banco
```

Se verificó la operatividad del motor de contenedores en el sistema anfitrión ejecutando `docker compose version`, obteniendo como respuesta la versión `v5.5.1` sobre Docker Desktop (Linux Engine 29.8.0), confirmando un entorno listo para la orquestación.

2. Paso 2: Despliegue con Compose y Diagnóstico de Salud

Se levantó la infraestructura completa mediante el comando oficial de compilación y ejecución desacoplada:

```
docker compose up --build -d
docker compose ps
```

Salida obtenida en consola:

```
NAME               IMAGE               COMMAND                  SERVICE   CREATED          STATUS
                   PORTS
mexi-banco-app-1   mexi-banco-app      "java -jar app.jar"      app       27 seconds ago   Up 20
    seconds (healthy)   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1    postgres:17-alpine  "docker-entrypoint.s..." db        27 seconds ago   Up 26
    seconds (healthy)   5432/tcp
```

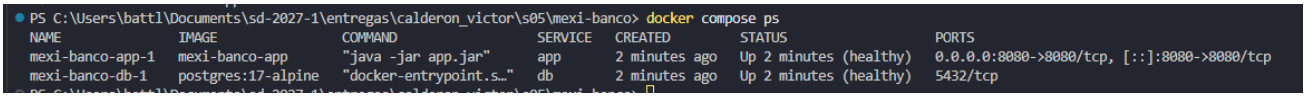


Figura 1: `docker compose ps`: `db` y `app` en estado `(healthy)`.

Observación técnica: Como estipula la guía, ambos servicios reportaron el estado `(healthy)` y no meramente *running*. El contenedor `db` validó la recepción efectiva de conexiones relacionales mediante el comando `pg_isready -U mexibanco -d mexibanco` cada 5 segundos. Por su parte, el contenedor `app` esperó a que la base de datos alcanzara este estado mediante `depends_on: { db: { condition: service_healthy } }`, evitando fallos transitorios en el pool de conexiones HikariCP, y acreditó su propia salud mediante sondas HTTP periódicas al endpoint de Actuator.

3. Paso 3: Validación Exhaustiva de Endpoints (Colección Postman)

Se importó la colección oficial `mexi-banco-v05.1.postman_collection.json` y se ejecutaron las cinco secciones de pruebas funcionales en orden estricto.
Nota sobre las CLABEs: la colección genera la CLABE al azar cada vez que se abren las cuentas (y la `Idempotency-Key` a partir de ella). En esta corrida quedaron `002180792225562971` para Ana, `012180792225562971` para Beto y la clave `prueba-792225562971`; son las que aparecen en todas las salidas y capturas de este reporte.

3.1. 1. Arriba y Respondiendo

Salud de la aplicación:

```
GET /actuator/health HTTP/1.1
Host: localhost:8080

HTTP/1.1 200 OK
Content-Type: application/vnd.spring-boot.actuator.v3+json

{"status":"UP","groups":["liveness","readiness"]}
```

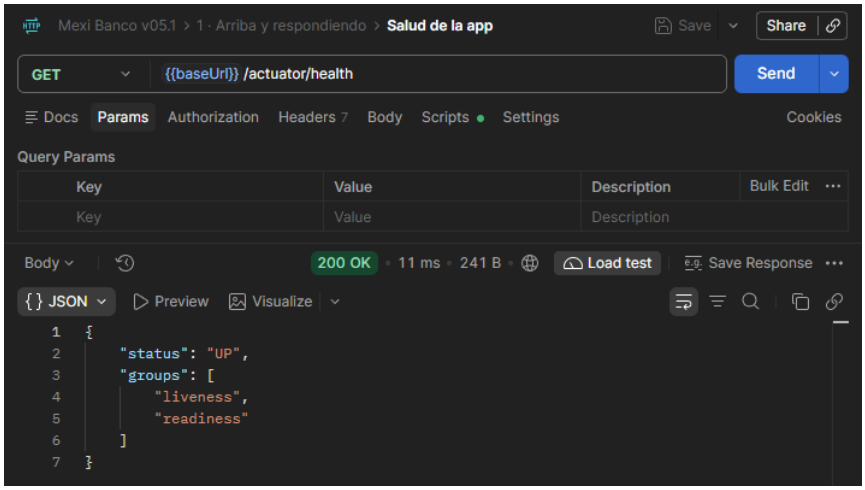


Figura 2: Postman: `GET /actuator/health` responde 200 con `status UP`.

Apertura y consulta de cuenta origen (Ana, $1,000.00):

```
POST /cuentas HTTP/1.1
Content-Type: application/json

{"clabe":"002180792225562971","titular":"Ana","saldoInicial":1000}

HTTP/1.1 201 Created
{"id":1,"clabe":"002180792225562971","titular":"Ana","saldo":1000}

GET /cuentas/002180792225562971 -> HTTP/1.1 200 OK | {"id":1,"clabe":"002180792225562971","titular":"
    Ana","saldo":1000.00}
```

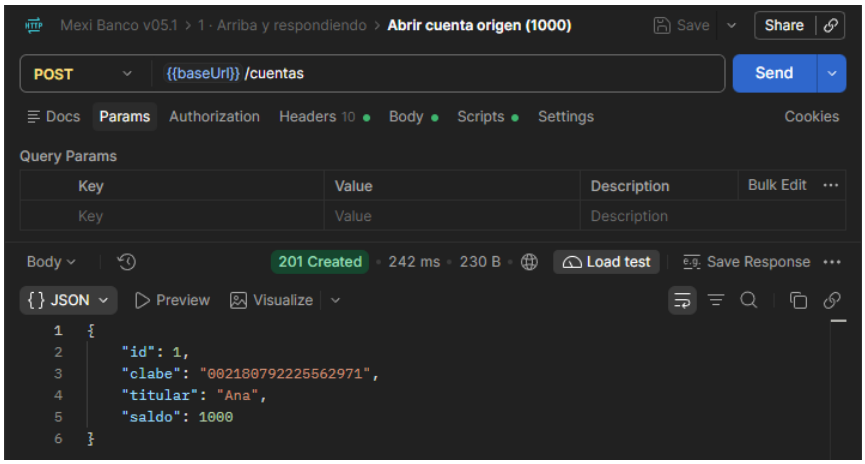


Figura 3: Postman: apertura de la cuenta de Ana con $1000 (201 Created).

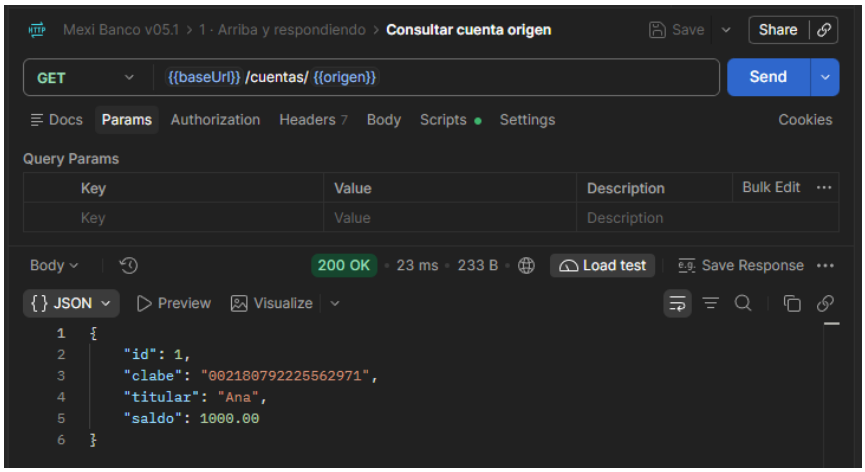


Figura 4: Postman: consulta de la cuenta de Ana (200, saldo 1000.00).

3.2. 2. Transferencia Interna Contable

Se abrió la cuenta destino para Beto con $500.00 y se transfirieron $200.00 desde la cuenta de Ana:

```
POST /cuentas HTTP/1.1
Content-Type: application/json
```

```
{"clabe":"012180792225562971","titular":"Beto","saldoInicial":500}

HTTP/1.1 201 Created
{"id":2,"clabe":"012180792225562971","titular":"Beto","saldo":500}

POST /transferencias HTTP/1.1
Content-Type: application/json

{"claveOrigen":"002180792225562971","claveDestino":"012180792225562971","monto":200}

HTTP/1.1 201 Created
{"id":1,"claveOrigen":"002180792225562971","claveDestino":"012180792225562971","monto":200,"fecha
    ":"2026-10-17T08:27:05.094146191Z"}
```

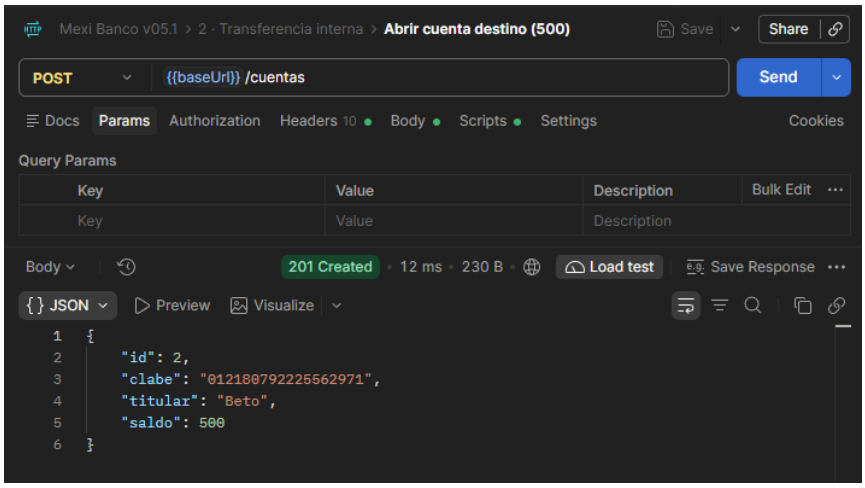


Figura 5: Postman: apertura de la cuenta de Beto con $500 (201 Created).

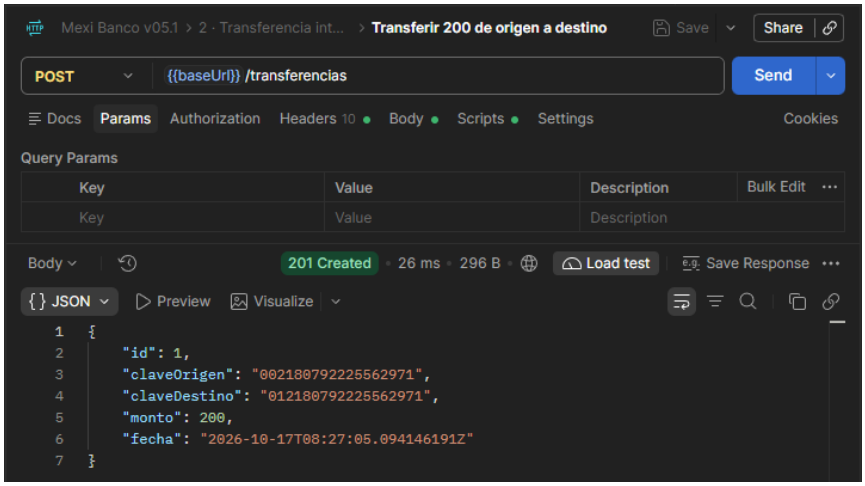


Figura 6: Postman: transferencia de $200 de Ana a Beto (201 Created).

La verificación de saldos demostró la cuadratura contable exacta: Ana quedó con $800.00 y Beto con $700.00:

```
GET /cuentas/002180792225562971 -> HTTP/1.1 200 OK | {"id":1,"clabe":"002180792225562971","titular":"
    Ana","saldo":800.00}
```

```
GET /cuentas/012180792225562971 -> HTTP/1.1 200 OK | {"id":2,"clabe":"012180792225562971","titular":"
    Beto","saldo":700.00}
```

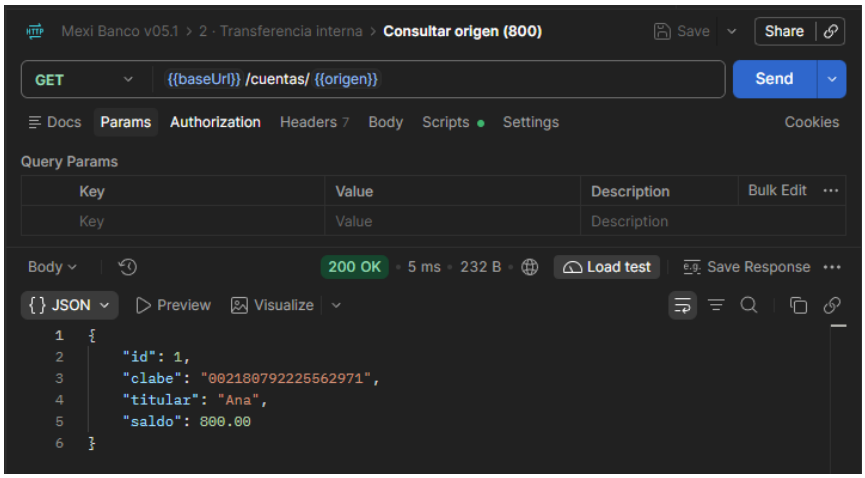


Figura 7: Postman: Ana queda con 800.00.

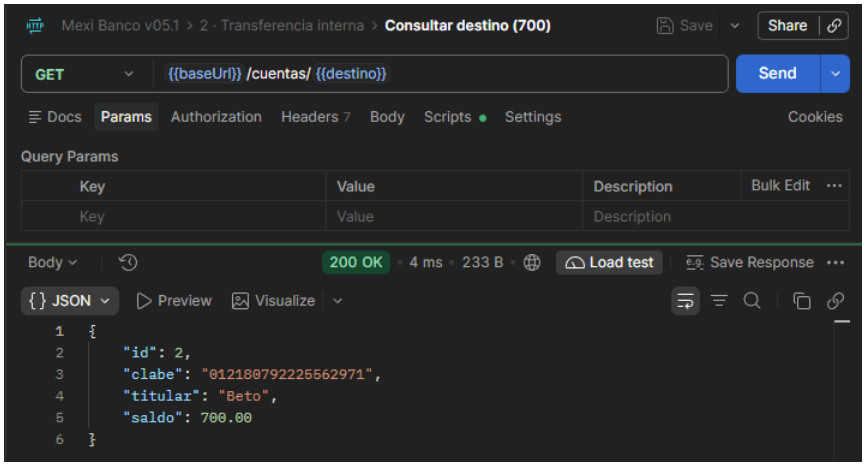


Figura 8: Postman: Beto queda con 700.00.

3.3. 3. Idempotencia en Envíos SPEI

Se probó el escenario crítico de duplicación enviando dos peticiones idénticas con la cabecera `Idempotency-Key: prueba-792225562971` por un monto de $50.00:

Primer envío:

```
POST /spei HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Idempotency-Key: prueba-792225562971

{"claveOrigen":"002180792225562971","bancoDestino":"BBVA","claveDestino":"012180000000000099","monto
    ":50}

HTTP/1.1 201 Created
```

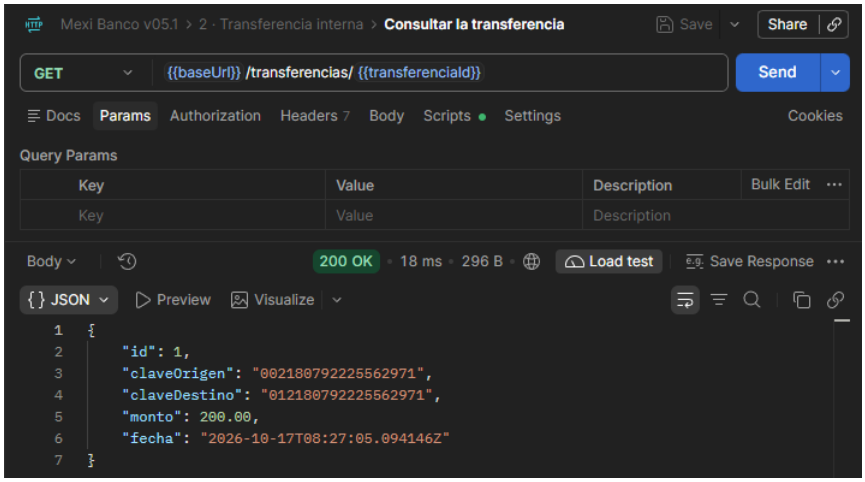


Figura 9: Postman: consulta de la transferencia registrada (`GET /transferencias/1`).

```
{"id":1,"idempotencyKey":"prueba-792225562971","claveOrigen":"002180792225562971","bancoDestino":"BBVA
    ","claveDestino":"012180000000000099","monto":50,"estado":"ENVIADO"}
```

Reintento inmediato (misma clave):

```
POST /spei HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Idempotency-Key: prueba-792225562971

{"claveOrigen":"002180792225562971","bancoDestino":"BBVA","claveDestino":"012180000000000099","monto
    ":50}

HTTP/1.1 201 Created
{"id":1,"idempotencyKey":"prueba-792225562971","claveOrigen":"002180792225562971","bancoDestino":"BBVA
    ","claveDestino":"012180000000000099","monto":50.00,"estado":"ENVIADO"}
```

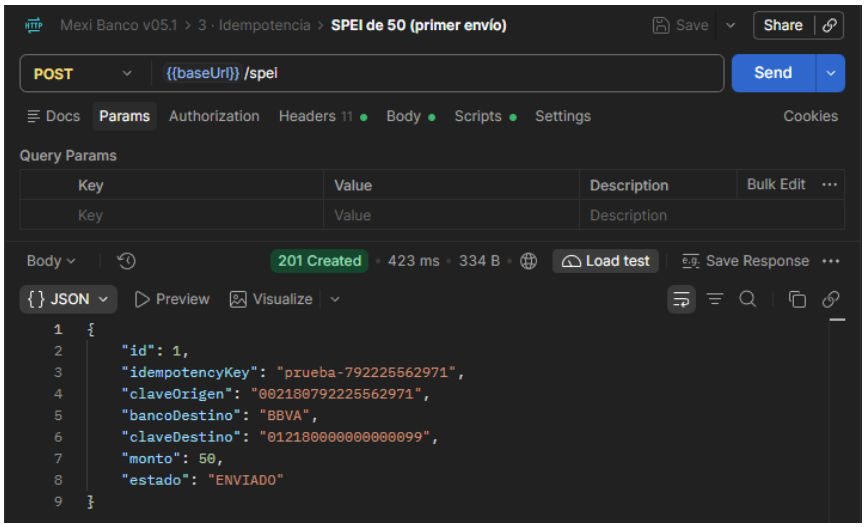


Figura 10: Postman: primer envío SPEI de \$50 (201 Created, `id` 1).

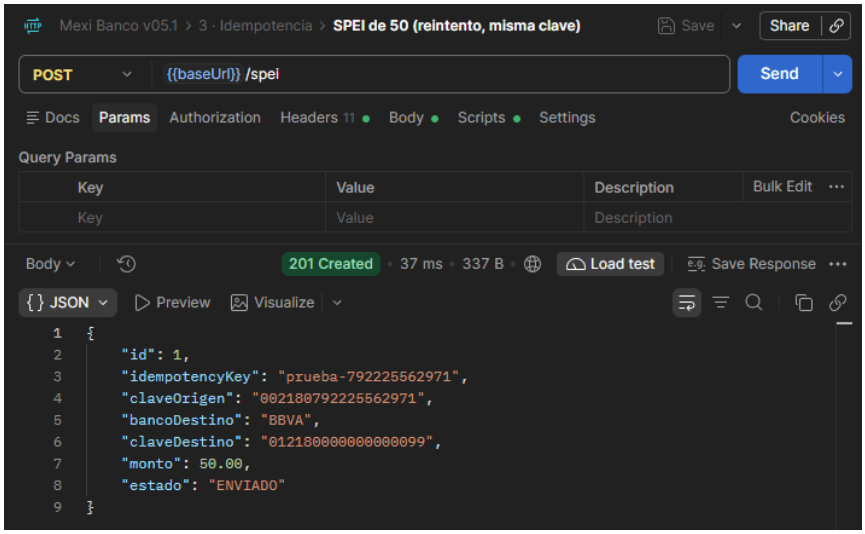


Figura 11: Postman: reintento con la misma `Idempotency-Key`; devuelve el mismo `id` 1 sin crear otro SPEI.

La consulta de la cuenta de origen confirmó que el saldo final fue de $750.00 (se debitaron $50.00 una sola vez, comprobando que el sistema no cobró doble):

```
GET /cuentas/002180792225562971 -> HTTP/1.1 200 OK | {"id":1,"clabe":"002180792225562971","titular":"
    Ana","saldo":750.00}
```

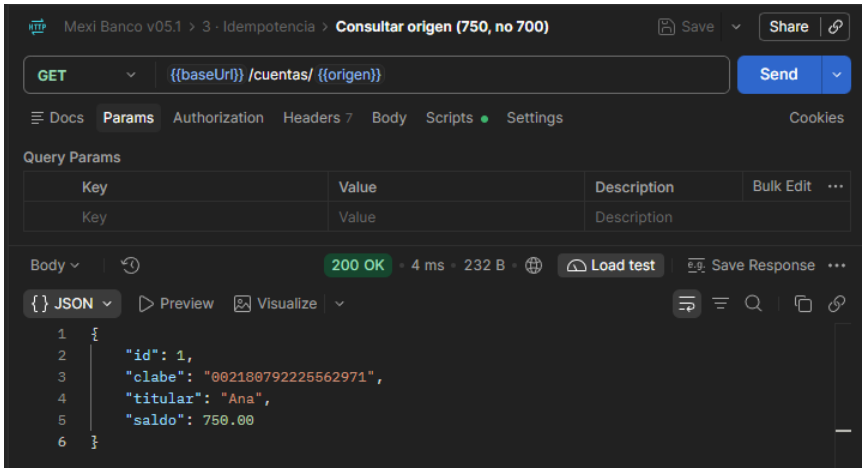


Figura 12: Postman: Ana queda con 750.00 (el SPEI se cobró una sola vez).

3.4. 4. El Monolito por Dentro: Estado de Cuenta y Avisos

Se consultaron los movimientos acumulados y las notificaciones emitidas para la CLABE de origen:

```
GET /movimientos?clabe=002180792225562971 HTTP/1.1
HTTP/1.1 200 OK
[
  {"id":5,"clabeCuenta":"002180792225562971","tipo":"SPEI_ENVIADO","monto":-50.00,"saldoResultante
    ":750.00,"fecha":"2026-10-17T08:28:10.119904Z","referencia":"BBVA · 012180000000000099"},
```

```
  {"id":3,"clabeCuenta":"002180792225562971","tipo":"TRANSFERENCIA_ENVIADA","monto":-200.00,"
    saldoResultante":800.00,"fecha":"2026-10-17T08:27:05.092713Z","referencia":"a 012180792225562971"},
  {"id":1,"clabeCuenta":"002180792225562971","tipo":"DEPOSITO","monto":1000.00,"saldoResultante
    ":1000.00,"fecha":"2026-10-17T08:26:03.218226Z","referencia":"saldo inicial"}
]

GET /notificaciones?clabe=002180792225562971 HTTP/1.1
HTTP/1.1 200 OK
[
  {"id":3,"clabeCuenta":"002180792225562971","mensaje":"SPEI_ENVIADO por 50, saldo resultante 750.00","
    fecha":"2026-10-17T08:28:10.524398Z"},
  {"id":1,"clabeCuenta":"002180792225562971","mensaje":"TRANSFERENCIA_ENVIADA por 200, saldo resultante
     800.00","fecha":"2026-10-17T08:27:05.096025Z"}
]
```

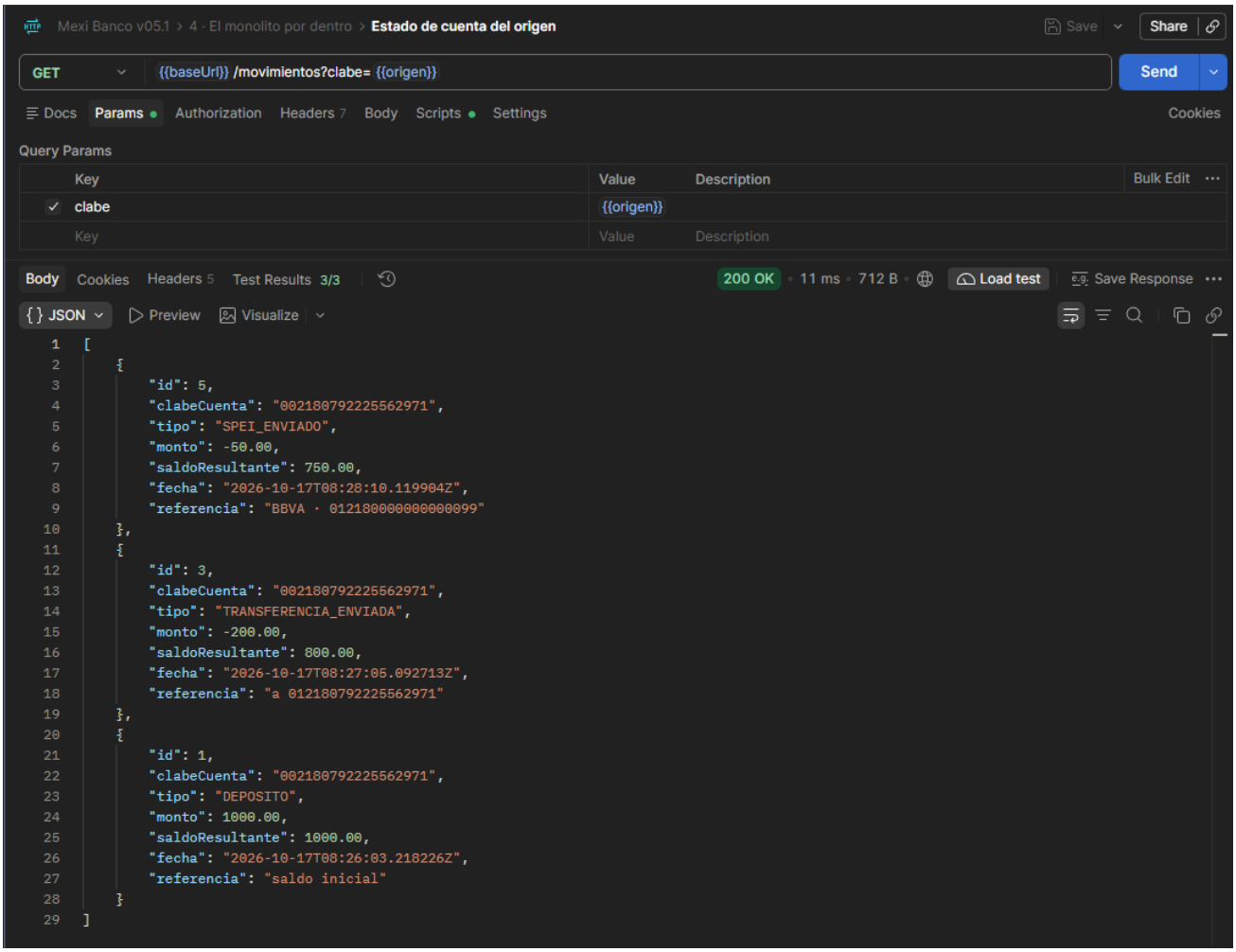



Figura 13: Postman: estado de cuenta de Ana (`GET /movimientos`), tres movimientos.

Observación: La suma algebraica de los tres movimientos ($1000 - 200 - 50 = 750$) cuadra al centavo con el saldo de la entidad, y se emitió exactamente una notificación por cada operación realizada.

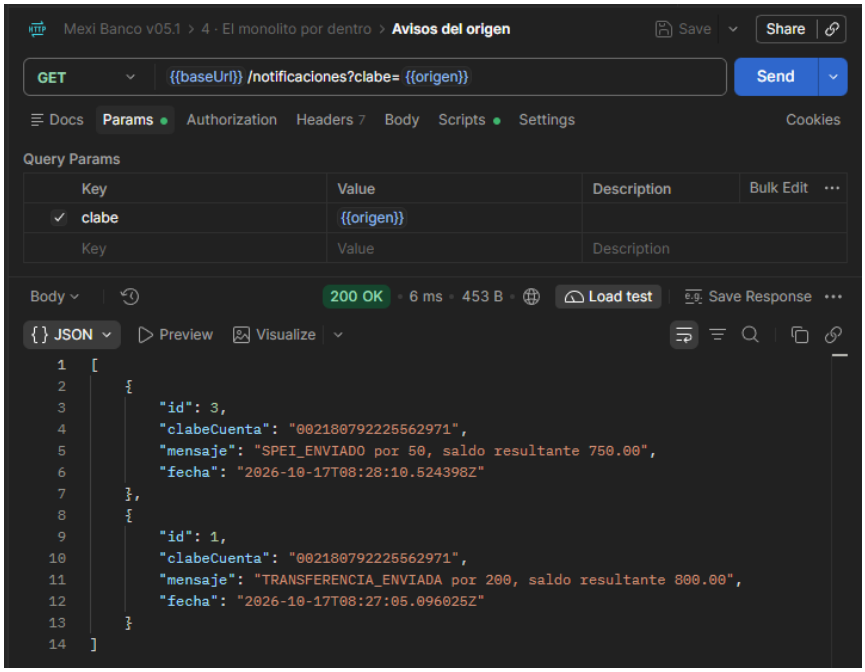


Figura 14: Postman: avisos de Ana (`GET /notificaciones`), uno por operación.

3.5. 5. Cuando Algo Sale Mal: Manejo Riguroso de Errores

Se sometió la API a las condiciones anómalas definidas en la colección, validando que cada situación devuelva el código semántico HTTP apropiado sin generar excepciones no capturadas (código 500):

Escenario de Prueba	**Endpoint / Causa**	**Código**	**Respuesta Semántica**
CLABE que no existe	`GET /cuentas/999999999999999999`	404	Entidad no encontrada
CLABE repetida	`POST /cuentas` (misma CLABE)	409	Conflicto de unicidad
Cuerpo sin CLABE	`POST /cuentas` (campo faltante)	400	Violación de validación
Saldo insuficiente	`POST /transferencias` (monto 999999)	422	Regla de negocio no procesable
Misma cuenta	`POST /transferencias` (origen == destino)	422	Transferencia redundante inválida
SPEI sin clave	`POST /spei` sin `Idempotency-Key`	400	Falta la cabecera obligatoria

Cuadro 1: Validación de códigos de error de negocio en la API (sin errores 500).

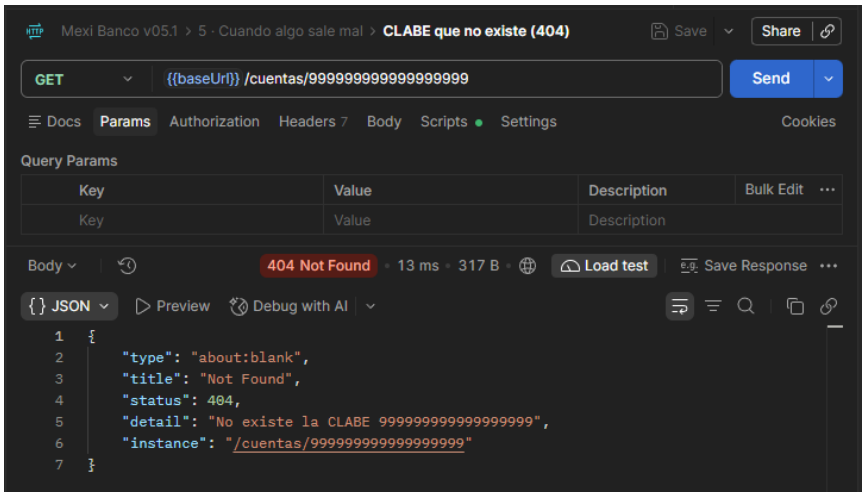


Figura 15: Postman: CLABE que no existe (404).

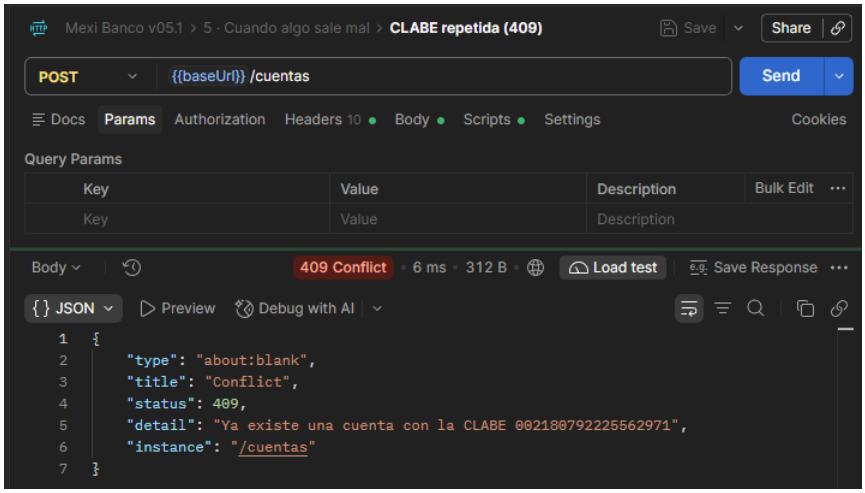


Figura 16: Postman: CLABE repetida (409).

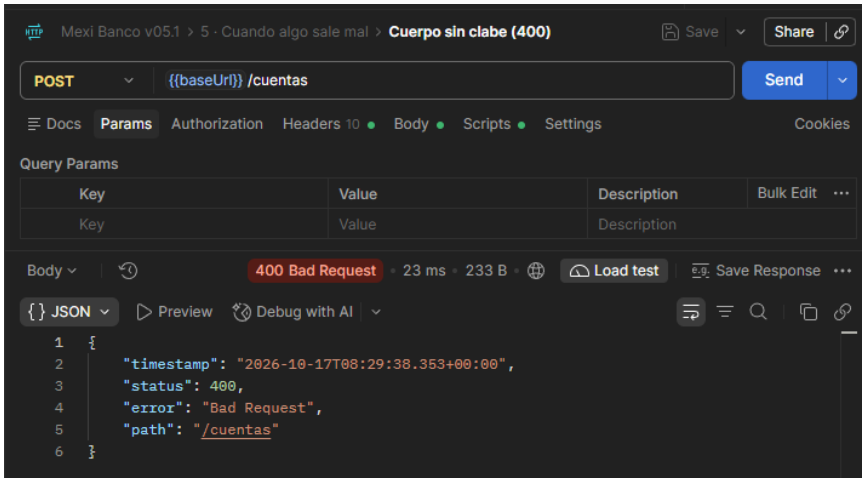


Figura 17: Postman: cuerpo sin CLABE (400).

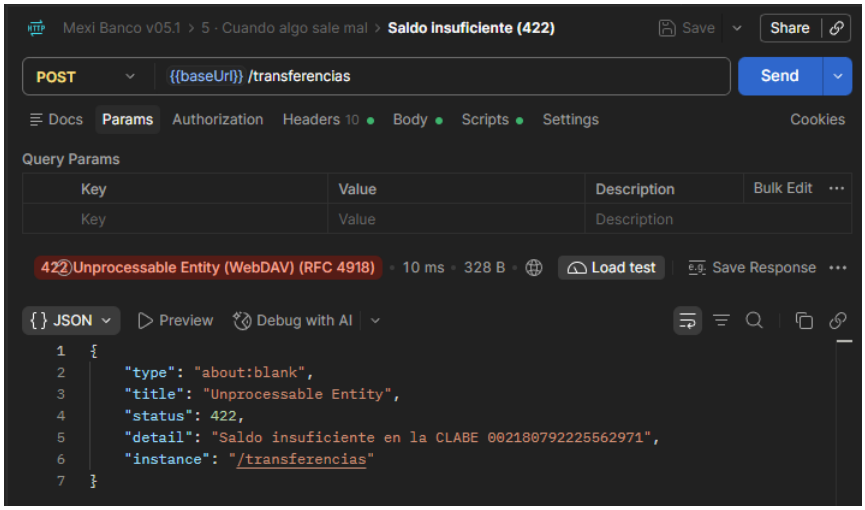


Figura 18: Postman: saldo insuficiente (422).

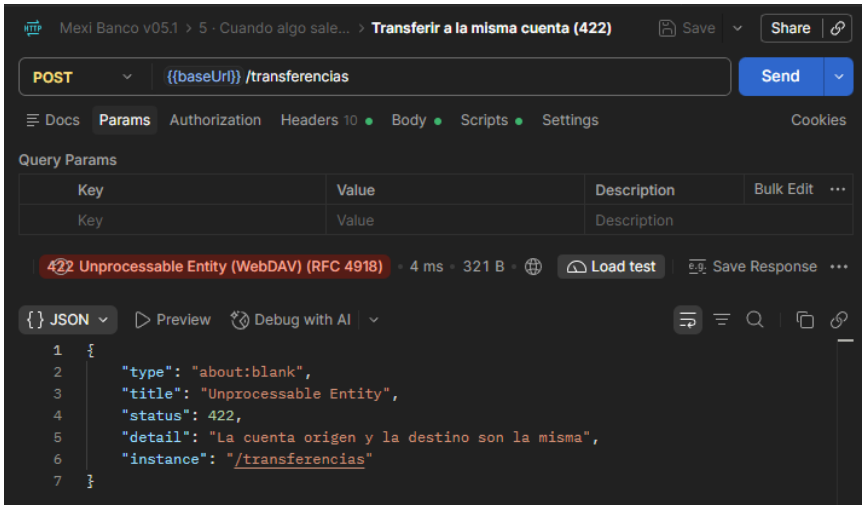


Figura 19: Postman: transferencia a la misma cuenta (422).

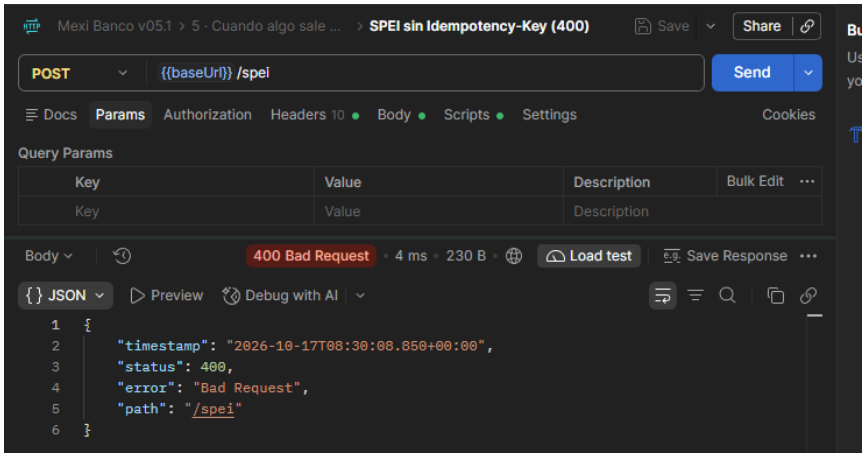


Figura 20: Postman: SPEI sin `Idempotency-Key` (400).

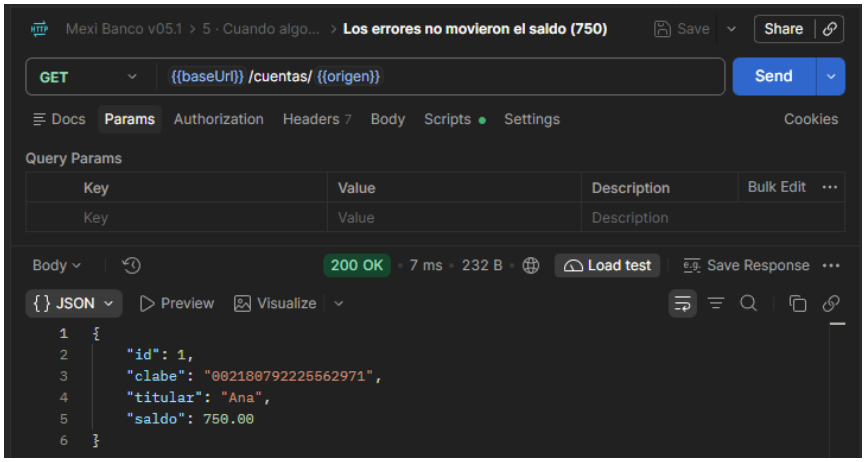


Figura 21: Postman: después de todos los errores el saldo de Ana sigue en 750.00.

4. Paso 4: Persistencia y Ciclo de Vida: Bájalo con y sin Volumen

La guía oficial instruye ejecutar dos experimentos destructivos para contrastar el comportamiento del almacenamiento gestionado por Docker.

4.1. Experimento A: Detención sin borrado de volumen (`down`)

```
docker compose down
docker compose up -d
curl.exe -i http://localhost:8080/cuentas/002180792225562971
```

Respuesta obtenida:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":1,"clabe":"002180792225562971","titular":"Ana","saldo":750.00}
```

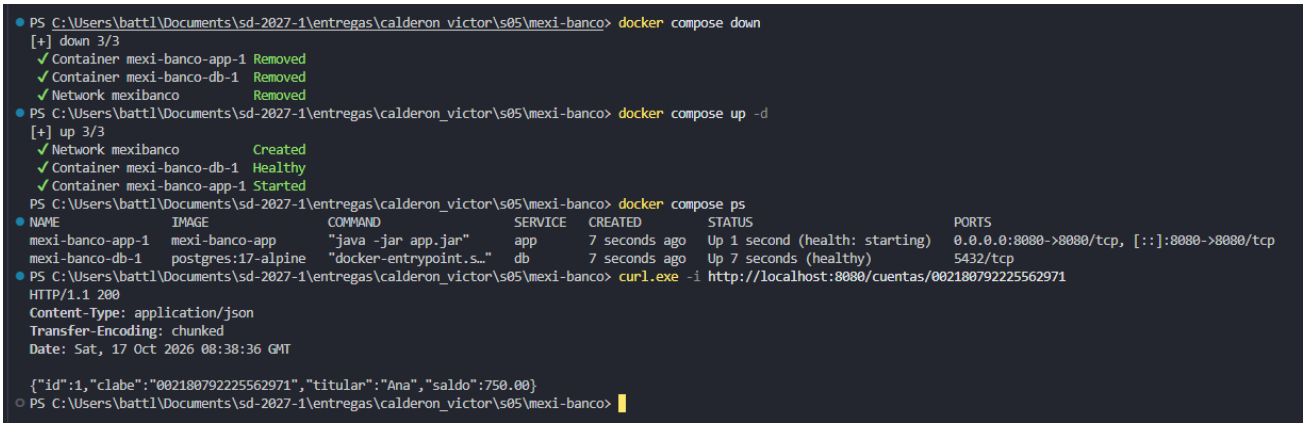


Figura 22: Terminal: `docker compose down`, `up -d` y consulta: la cuenta sigue ahí (200, saldo 750.00).

4.2. Experimento B: Detención borrando el volumen (`down -v`)

```
docker compose down -v
docker compose up -d
curl.exe -i http://localhost:8080/cuentas/002180792225562971
```

Respuesta obtenida:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE
    002180792225562971","instance":"/cuentas/002180792225562971"}
```

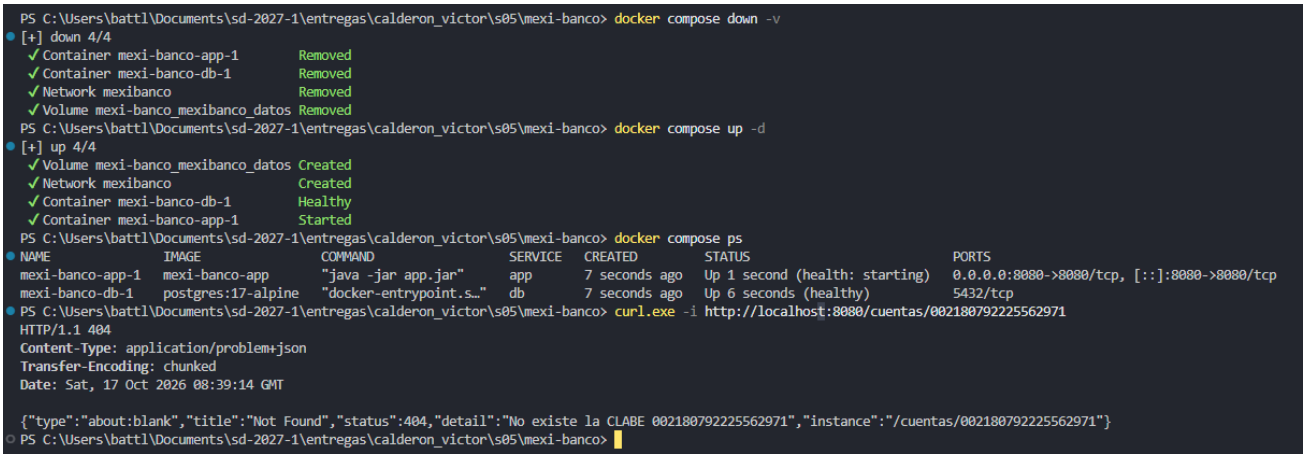


Figura 23: Terminal: `docker compose down -v` (se borra el volumen `mexi-banco_mexibanco_datos`), `up -d` y consulta: 404.

4.3. Respuesta Oficial: ¿Qué pasó con los datos en cada caso, y por qué?

- **En el caso de `docker compose down`:** Los datos de las cuentas, saldos y movimientos se conservaron íntegramente. Esto se debe a que la directiva `down` únicamente destruye los contenedores en ejecución y la red virtual, pero preserva los volúmenes nombrados definidos en la sección `volumes:` de Compose en el sistema de archivos del host (`/var/lib/docker/volumes/`). Cuando el contenedor `db` vuelve a levantarse con `up -d`, el motor monta nuevamente el volumen preexistente `mexibanco_datos`; PostgreSQL detecta que el clúster ya se encuentra inicializado y no ejecuta `initdb`, manteniendo la persistencia intacta.

- **En el caso de `docker compose down -v`:** Los datos se destruyeron irreversiblemente y la consulta arrojó `404 Not Found`. La bandera `-v` (*–volumes*) le instruye explícitamente al daemon de Docker que purgue los volúmenes asociados al proyecto además de los contenedores. Al levantarse el entorno de nuevo, PostgreSQL se encuentra con un directorio montado completamente vacío, ejecuta el script de inicialización de clúster desde cero y Spring Boot inicializa tablas vírgenes según su directiva `ddl-auto: update`, perdiéndose la totalidad de las cuentas preexistentes.

5. Paso 5: Revisión de la Arquitectura del Código Fuente

Siguiendo las indicaciones del laboratorio, se inspeccionó la estructura interna del proyecto en `src/main/java/mx/mexibanco`. El sistema está concebido como un monolito modular con cinco dominios: `cuenta`, `movimiento`, `transferencia`, `notificacion` y `spei`. Cada uno de ellos implementa una separación limpia en cuatro capas arquitectónicas:

1. **Controlador (*Controller*):** Expone los endpoints REST, recibe las solicitudes HTTP, aplica validaciones formales con anotaciones Jakarta (`@NotBlank`, `@Positive`) y transforma las excepciones de negocio en códigos HTTP mediante el manejador global.
2. **Servicio (*Service*):** Coordina la lógica de negocio y aplica las reglas de dominio (por ejemplo, validación de saldos y cálculo de montos).
3. **Repositorio (*Repository*):** Interfaz que extiende `JpaRepository`, encargada del mapeo objeto-relacional y de ejecutar consultas SQL atómicas.
4. **Entidad (*Entity*):** Clases JPA que modelan el esquema relacional en PostgreSQL (`Cuenta`, `Movimiento`, `Transferencia`, `SpeiSolicitud`).

5.1. El Patrón de Idempotencia y Aislamiento Transaccional

El diseño de `SpeiService.java` y `SpeiTransacciones.java` sobresale por su rigurosidad técnica:

```
// SpeiService.java
public SpeiSolicitud procesar(String idempotencyKey, SolicitudSpei datos) {
    SpeiSolicitud reservada;
    try {
        reservada = transacciones.reservar(idempotencyKey, datos);
    } catch (DataIntegrityViolationException choqueDeUnique) {
        reservada = null;
    }
    if (reservada != null) {
        return transacciones.ejecutar(reservada);
    }
    SpeiSolicitud existente = solicitudes.findByIdempotencyKey(idempotencyKey)
        .orElseThrow(() -> new IllegalStateException("La reserva choco pero no aparece"));
    if (existente.getEstado() == EstadoSpei.PROCESANDO) {
        throw new ConflictoException("Ya hay un SPEI en vuelo con esta Idempotency-Key");
    }
    return existente;
}
```

La separación en dos beans es deliberada: Spring gestiona transacciones mediante *proxies* dinámicos; invocar un método transaccional desde la misma clase omite el interceptor. Al delegar la operación en el componente `SpeiTransacciones` con propagación `REQUIRES_NEW`, la inserción de la clave con estado `PROCESANDO` se confirma en su propia transacción atómica antes de cobrar.
Si un reintento ingresa con la misma clave, la restricción `UNIQUE` en la columna de la llave dispara la excepción de integridad. `SpeiService` atrapa el error fuera de la transacción fallida, consulta el registro existente y devuelve el resultado previo sin tocar los saldos ni duplicar movimientos contables.

Paso	Tiempo	Actor / Componente	Acción Ejecutada	Estado BD / Respuesta
1	t_0	Cliente HTTP	`POST /spei` (`spei-001`)	Petición recibida en controlador
2	t_1	`SpeiTransacciones.reservar`	`INSERT` en `spei_solicitud`	Clave guardada (`PROCESANDO`)
3	t_2	`SpeiTransacciones.ejecutar`	Débito \$50 en cuenta `002180`	Saldo pasa de \$800 a \$750
4	t_3	`SpeiTransacciones.ejecutar`	`INSERT` en `movimiento`	Fila `SPEI_ENVIADO` (\$ -50)
5	t_4	`SpeiTransacciones.ejecutar`	`UPDATE` a estado `ENVIADO`	Transacción confirmada en BD
6	t_5	Cliente HTTP	Respuesta 201 Created	Retorno de `SpeiSolicitud` (ID 1)
7	t_6	Cliente HTTP (Reintento)	`POST /spei` (`spei-001`)	Petición duplicada por timeout
8	t_7	`SpeiTransacciones.reservar`	Intento de inserción repetido	`DataIntegrityViolationException`
9	t_8	`SpeiService`	Captura error y busca clave	Registro hallado en estado `ENVIADO`
10	t_9	`SpeiService`	Retorna entidad existente	Respuesta 201 con ID original

Cuadro 2: Interleaving temporal: manejo de idempotencia y prevención de cobro duplicado.

6. Paso 6: Estudio del Compose y del Dockerfile

6.1. Análisis Línea por Línea de `docker-compose.yml`

```
services:
  db:
    image: postgres:17-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5
```

```
networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

Desglose de componentes:

1. **Servicios levantados:** `db` levanta la imagen oficial de PostgreSQL 17 optimizada sobre Alpine Linux. `app` construye la imagen local a partir del contexto actual mediante `build: .`.
2. **Resolución y red:** Ambos servicios se asocian a la red nombrada `mexibanco`. El servicio `app` resuelve a la base de datos utilizando el hostname virtual `db` mediante el DNS embebido de Docker (`127.0.0.11`), sin requerir acoplamiento a direcciones IP estáticas.
3. **Aislamiento de puertos:** El puerto 5432 de PostgreSQL no se publica al host físico; únicamente `app` expone `8080:8080`, aplicando el principio de mínima exposición de superficie.
4. **Orquestación y Healthchecks:** `depends_on` con condición `service_healthy` evita que Spring Boot inicie en frío antes de que PostgreSQL acepte conexiones SQL válidas.
5. **Persistencia:** El volumen nombrado `mexibanco_datos` desacopla el almacenamiento de los datos de la vida útil del contenedor de base de datos.

6.2. Análisis Línea por Línea de `Dockerfile`

```
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN chmod +x ./mvnw && ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Desglose de etapas:

1. **Etapa 1 (Compilación / Builder):** Utiliza un JDK 21 completo sobre Alpine. Copia primero los descriptores del proyecto (`pom.xml`, `mvnw`, `.mvn`) y ejecuta `dependency:go-offline`. Esta técnica optimiza la caché de capas de Docker: si el código fuente cambia pero las dependencias permanecen iguales, Docker reutiliza la capa descargada. Posteriormente, copia el código fuente (`src/`) y compila el paquete `.jar`.
2. **Etapa 2 (Ejecución / Runtime):** Se basa en un JRE 21 ligero sobre Alpine. Copia únicamente el binario ejecutable desde la etapa previa (`COPY -from=builder`). No incluye compiladores, código fuente ni el repositorio local de Maven, reduciendo drásticamente el tamaño final de la imagen y fortaleciendo la seguridad en producción.

7. Conclusiones

El Laboratorio S05 evidenció cómo Docker Compose permite estandarizar entornos de desarrollo distribuidos de alta fidelidad con respecto a producción. El uso riguroso de sondas de salud garantiza la resiliencia en la inicialización de dependencias, mientras que la separación entre volúmenes y contenedores permite gestionar el estado de forma determinista. Finalmente, el análisis del monolito demostró que la modularidad interna en capas y el uso de transacciones independientes con restricciones de unicidad son la base fundamental para garantizar la idempotencia antes de dar el paso hacia una descomposición en microservicios y despliegues en Kubernetes.